

Campus Log Anomaly Detection under Proxy Labels: Validity Limits of Archive Splits and Masked TCN–Transformers

Thanh Tien Cao^{1*}, Tu Hoang Trinh¹, and Ha Manh Tran¹

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages – Information Technology,
Ho Chi Minh City, Vietnam
thanhct@huflit.edu.vn, 23dh113972@st.huflit.edu.vn, hatm@huflit.edu.vn

Abstract. Campus log anomaly studies often treat archive order as time and window OR-aggregation as a label. We measure how those choices distort conclusions on a selective HUFLIT stream (1,000,000 deduplicated lines; custom similarity parser; HTTP/keyword *proxy* labels). On a leaky stream-position split, test-window prevalence reaches 80.5% because a window is positive if any of $L=20$ lines is positive; an always-anomaly classifier then obtains $F1 \approx 0.892$, while a masked TCN–Transformer under $\tau = \mu + 3\sigma$ scores only 0.0395 ± 0.0163 F1 despite ROC-AUC 0.7226 ± 0.0178 (AUPRC lift over prevalence $\approx +0.031$). After global line-hash deduplication, source/member-bounded windows, and a day-held-out split (train 15–17 Jun, val 18 Jun, test 19 Jun 2026), test prevalence falls to 56.4% (always-anomaly F1 0.721), yet masked models collapse to near-chance ranking (TCN–Transformer ROC-AUC 0.5315 ± 0.112 ; no-TCN 0.4933 ± 0.009), while PCA retains a modest AUPRC lift (+0.098). Threshold sweeps show $\mu + 3\sigma$ suppresses recall; looser rules inflate F1 toward the trivial baseline. The TCN adds latency (batch-1 51.1 vs. 18.5 ms) without a consistent accuracy benefit. Local LLM RCA is left to future work.

Keywords: AIOps · Log Anomaly Detection · Proxy Labels · Experimental Validity · TCN–Transformer · Campus Infrastructure.

1 Introduction

Campus IT services emit heterogeneous operational logs. Sequence detectors can score unusual template patterns [2–5], but archive packaging, duplicate backups, and heuristic labels can invent “temporal” structure that models then appear to learn.

This paper treats those validity threats as first-class experimental objects. We study a masked TCN–Transformer and a no-TCN ablated Transformer on HUFLIT selective extracts, with two protocols: (A) a leaky stream-position split

* Corresponding author: thanhct@huflit.edu.vn

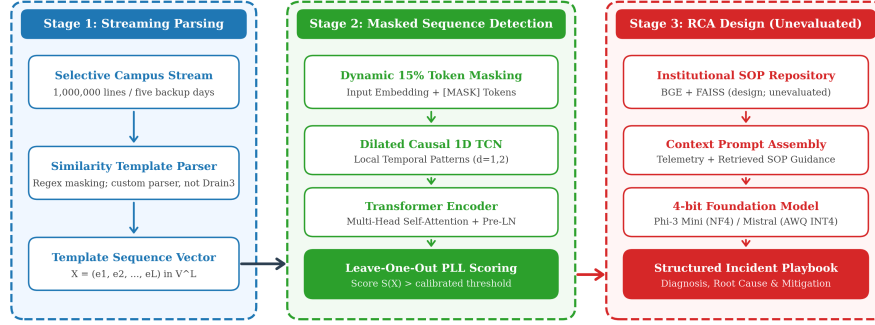


Fig. 1. Architectural overview of the measured detector (parsing + masked PLL scoring). Local LLM RCA is deferred.

that mirrors a common but flawed archive ordering; (B) a deduplicated, source-bounded, day-held-out split. We add trivial baselines (always-anomaly; no-skill AUPRC = prevalence), report AUPRC lift over prevalence, and sweep validation thresholds. We do *not* claim an LLM root-cause system; local narration is deferred.

The contributions are:

- **Validity protocol:** quantify how OR window labels and archive-order splits inflate prevalence and F1 ceilings; release dual-split artifacts (`huflit_baselines.json`, `huflit_v2.*`).
- **Dedup + day split:** global SHA-1 line dedup, windows that never cross (day, source, member), train/val/test by backup day.
- **Threshold-aware reading:** $\mu + k\sigma$ and percentile sweeps plus a validation-oracle diagnostic, separating ranking from calibration.
- **Negative TCN evidence:** on both protocols the TCN does not consistently beat a masked Transformer without TCN, while increasing leave-one-out scoring latency.

2 Related Work

2.1 Log Parsing and Template Extraction

Log parsing transforms free-form messages into event templates by separating constants from dynamic parameters [1, 17]. Drain uses a fixed-depth parse tree [1]; Spell uses streaming sequence matching [17]. Our artifact does *not* implement Drain or Drain3. It applies regex masking and a linear scan over equal-length templates, merging tokens when positional similarity exceeds 0.5. We call this the *custom similarity parser* and avoid attributing Drain3 properties or throughput to it.

2.2 Deep Learning for Log Anomaly Detection

Sequence-based anomaly detection models learn temporal transition dynamics over sequences of log templates [2, 3]. Du et al. [2] introduced DeepLog, employing Long Short-Term Memory (LSTM) networks to model normal log sequences and flag unexpected subsequent templates as anomalous. Meng et al. [3] proposed LogAnomaly, integrating semantic template vectors via FastText to capture semantic similarities. SwissLog [5] and LogRobust [4] further address log instability and interleaved execution threads. While deep sequence architectures achieve strong detection precision, they operate as black-box scoring engines: scalar anomaly metrics fail to explain the operational root cause, imposing heavy manual triage overhead on operators.

2.3 Large Language Models in Log Analytics and RCA

Recent work includes Transformer-TCN detection [24], masked/contrastive log models [22], and LLM-assisted detectors such as KRONE [19], MaidLog [20], CoLA [25], and Smart Eye [23]. LLM4Log [18] surveys the area; cloud RCA recommenders are studied in [13, 14]. Our focus is experimental validity on campus archives rather than a new LLM narrator.

Table 1. Qualitative comparison of related systems (capabilities as published). Latency is *not* cross-comparable: this work reports batch-1 leave-one-out PLL wall time; DeepLog/LogAnomaly-lite rows use legacy batched scoring (NR for true batch-1).

Approach	Parser	Detector	RCA	Local LLM RCA	CPU ms
DeepLog [2]	Spell	LSTM next-token	Scalar score	×	NR [†]
LogAnomaly [3]	Template+FT	Semantic LSTM	Scalar score	×	NR ^{†,‡}
LogRobust [4]	Drain	Bi-LSTM+Attn	Scalar score	×	NR
SwissLog [5]	Online	Attentive Bi-LSTM	Heuristic loc.	×	NR
KRONE [19]	LLM hierarchy	Hybrid+LLM	Nested LLM	Cloud/API-oriented	NR
This work	Custom similarity	Masked TCN-Transformer	Deferred	×	51.1 [§]

[†]Legacy batched ms retained in JSON but not relabeled as batch-1 PLL. [‡]LogAnomaly-lite reimplementations on our templates, not the authors’ FastText stack. [§]Batch-1 leave-one-out PLL on the deduplicated day-split (CPU, 2 threads).

3 Methodology

3.1 System Architecture Overview

The measured system has two stages: (i) **Parsing**: selected syslog messages are tokenized by the custom similarity parser; (ii) **Masked sequence scoring**: sliding template windows are corrupted by true token replacement, encoded by an optional causal TCN plus Transformer, and scored by leave-one-position-out pseudo-log-likelihood (PLL). Local LLM+RAG narration is *out of scope* for this revision (Future Work).

3.2 Masked TCN-Transformer Denoising Sequence Model

We formulate the detector as a masked template language model: 15% of input tokens are replaced by a reserved mask token, representations pass through a Temporal Convolutional Network (TCN) [7] and Transformer self-attention [8], and a linear vocabulary head predicts the original identities only at selected positions.

Embedding and Dynamic Masking A sliding window of length $L = 20$ with stride $S = 10$ (as used in the measured HUFLIT runs) extracts template sequences $\mathbf{X} = [T_1, T_2, \dots, T_L]$. During training, 15% of the template tokens are randomly replaced with a reserved [MASK] token ($ID = |\mathcal{V}| + 1$), distinguishing masked positions from sequence padding ($ID = 0$). Padding masks are applied across self-attention layers to prevent padded positions from corrupting contextual representations. Input tokens are embedded into a d_{model} -dimensional continuous vector space:

$$\mathbf{E} = \text{Embedding}(\mathbf{X}) + \mathbf{P}_{\text{pos}} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

where $\mathbf{P}_{\text{pos}}$ denotes sinusoidal positional encodings [8]:

$$\mathbf{P}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \mathbf{P}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

Temporal Convolutional Network (TCN) Layer To capture localized temporal patterns across consecutive log events, the embedded sequence $\mathbf{E}$ passes through a stack of 1D dilated causal convolutional layers [7]. For an input sequence $\mathbf{x} \in \mathbb{R}^L$ and a convolutional filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation with dilation factor d is defined as:

$$\mathbf{y}(t) = (\mathbf{x} *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}(t - d \cdot i) \quad (3)$$

We configure dilation factors $d \in \{1, 2\}$, enforcing causal convolutions for localized temporal extraction without future feature leakage. Residual connections and layer normalization ensure stable gradient propagation:

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\text{Proj}(\mathbf{E}) + \text{GELU}(\text{CausalTCN}(\mathbf{E}))) \quad (4)$$

Transformer Multi-Head Self-Attention Encoder The output $\mathbf{H}_{\text{TCN}}$ is then fed into a Transformer Encoder block to model long-range inter-template correlations. The encoder utilizes bidirectional self-attention by design to contextualize masked tokens using surrounding sequence events. For queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (5)$$

$$\text{MHA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (6)$$

$$\text{head}_i = \text{Attention}(\mathbf{H} \mathbf{W}_i^Q, \mathbf{H} \mathbf{W}_i^K, \mathbf{H} \mathbf{W}_i^V) \quad (7)$$

The Transformer block uses pre-layer normalization, residual connections, and a two-layer feed-forward network with GELU activation:

$$\text{FFN}(\mathbf{Z}) = \text{GELU}(\mathbf{Z} \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (8)$$

$$\mathbf{Z} = \mathbf{H} + \text{MHA}(\text{LayerNorm}(\mathbf{H})), \quad \mathbf{H}_{\text{out}} = \mathbf{Z} + \text{FFN}(\text{LayerNorm}(\mathbf{Z})). \quad (9)$$

Masked Reconstruction Objective and Anomaly Scoring A linear classification head projects $\mathbf{H}_{\text{out}}$ back to the template vocabulary space $\mathbb{R}^{|\mathcal{V}|}$, serving as the reconstruction decoder. The model is trained to predict original template identities at masked positions via Cross-Entropy loss:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log P(T_i = t_i \mid \mathbf{X}_{\setminus \mathcal{M}}) \quad (10)$$

During inference, we compute pseudo-log-likelihood by making one forward pass per valid position (batched over sequences), masking only that position and excluding padding ($ID = 0$):

$$S(\mathbf{X}) = \frac{1}{L_{\text{valid}}} \sum_{t=1}^{L_{\text{valid}}} -\log P(T_t = t_t \mid \mathbf{X}_{\setminus \{t\}}) \quad (11)$$

A validation-calibrated threshold τ flags anomalous sequences:

$$\tau = \mu_{\text{val}} + 3 \cdot \sigma_{\text{val}} \quad (12)$$

where μ_{val} and σ_{val} are the mean and standard deviation of scores over nominal validation sequences. We additionally report $k \in \{1, 2, 3\}$, nominal percentiles $\{95, 97.5, 99\}$, and a validation-oracle F1 threshold as a *diagnostic* upper bound (not a deployable claim).

4 Experimental Evaluation

4.1 Corpus and Window Labels

We stream selective Thang6 day zips from a local campus extract (`D:/huflit-campus-logs`), skipping `fortigate.csv` and members >500 MiB (no full RAR expand). Proxy line labels use HTTP 4xx/5xx, deny/block fields, and keywords—not SOC adjudication. A window of length $L=20$ (step 10) is positive iff *any* constituent line is positive (OR aggregation). If line anomaly rate is p , then under independence $1 - (1 - p)^L$ approximates window prevalence; with $p=0.0796$ this predicts ≈ 0.81 , matching the leaky split’s 80.5% test prevalence.

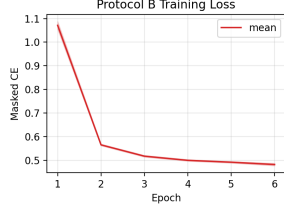


Fig. 2. Protocol B masked training loss (mean over seeds).

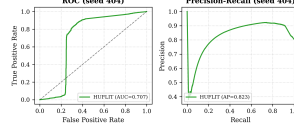


Fig. 3. Protocol A ROC/PR (leaky split; descriptive).

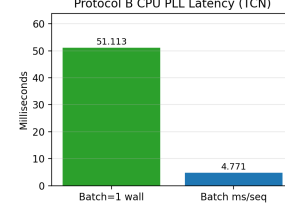


Fig. 4. Protocol B batch-1 vs batched PLL latency.

4.2 Protocol A (leaky archive split)

Records are ordered by backup day $\rightarrow$ ZIP $\rightarrow$ member $\rightarrow$ in-member position, then split 80/10/10 before windowing. This is *not* event time: windows may cross file/service boundaries, and duplicate backups leak across partitions (36.235% of unique validation lines also appear in training) [18]. Results live in `huflit_baselines.json`.

4.3 Protocol B (dedup + source-bounded + day split)

We keep the first SHA-1 occurrence of each raw line (262,194 duplicates skipped while filling 1M unique lines), emit windows only inside a single (day, source, member) group, and assign days 15–17 Jun $\rightarrow$ train, 18 Jun $\rightarrow$ val, 19 Jun $\rightarrow$ test. Day folders remain backup collection periods, not verified per-event timestamps. Sources kept are `aca/careerhub/courses`. Test-window prevalence drops to 56.36% (always-anomaly F1 0.721). Exact window-hash overlap shrinks to 505 train/val and 110 train/test shared windows (`huflit_v2_split_meta.json`).

4.4 Models and Metrics

Masked models use true 15% token replacement, nominal-only training, five seeds, and leave-one-out PLL. We report Precision/Recall/F1 at $\mu+3\sigma$, ROC-AUC, AUPRC, **AUPRC lift over prevalence**, always-anomaly F1, and batch-1 PLL latency (CPU, 2 threads). DeepLog/LogAnomaly rows in Protocol A are retained from the prior same-array suite for ranking context.

Table 2. Protocol A (leaky stream-position): $\tau = \mu+3\sigma$; test prevalence 80.5%; always-anomaly F1 $\approx$ 0.892; AUPRC no-skill $\approx$ 0.805

Model	F1	ROC-AUC	AUPRC	Lift	Batch-1 ms
Always-anomaly (trivial)	0.892	0.500	0.805	0.000	—
PCA	0.000 $\pm$ 0.000	0.738 $\pm$ 0.004	0.913 $\pm$ 0.003	+0.108	NR
DeepLog	0.011 $\pm$ 0.022	0.781 $\pm$ 0.020	0.882 $\pm$ 0.025	+0.077	NR
Masked Transformer (No TCN)	0.348 $\pm$ 0.410	0.769 $\pm$ 0.035	0.870 $\pm$ 0.036	+0.065	2.04
Masked TCN-Transformer	0.040 $\pm$ 0.016	0.723 $\pm$ 0.018	0.836 $\pm$ 0.014	+0.031	52.92

Table 3. Protocol B (dedup + source-bounded + day split): primary $\tau = \mu + 3\sigma$; test prevalence 56.4%; always-anomaly F1 0.721

Model	F1@3 σ	ROC-AUC	AUPRC	Lift	Batch-1 ms
Always-anomaly (trivial)	0.721	0.500	0.564	0.000	—
PCA	0.000 ± 0.000	0.629 ± 0.008	0.661 ± 0.006	+0.098	—
Masked Transformer (No TCN)	0.001 ± 0.002	0.493 ± 0.009	0.511 ± 0.007	-0.052	18.54 ± 0.47
Masked TCN-Transformer	0.151 ± 0.183	0.532 ± 0.112	0.557 ± 0.124	-0.007	51.11 ± 9.08

Table 4. Protocol B threshold sensitivity for Masked TCN-Transformer (mean F1 over 5 seeds; AUC unchanged by threshold)

$\mu+1\sigma$	$\mu+2\sigma$	$\mu+3\sigma$	$p95$	$p99$	Oracle-val
0.696	0.585	0.151	0.300	0.181	0.741

4.5 Results

On Protocol A, DeepLog leads ROC-AUC (0.781) while reconstruction AEs obtain higher F1 variance; the TCN is *worse* than no-TCN on mean F1/AUC and $\approx 26\times$ slower at batch-1 PLL. Critically, always-anomaly F1 (0.892) exceeds every learned F1 at $\mu+3\sigma$, and AUPRC values must be read against prevalence 0.805.

On Protocol B, masked models lose day-to-day ranking ($\text{AUC} \approx 0.49\text{--}0.53$), so prior archive-order gains do not survive dedup+day holdout. PCA keeps a small positive AUPRC lift (+0.098). Table 4 shows $\mu+3\sigma$ starves recall; $\mu+1\sigma$ F1 (0.696) approaches the trivial ceiling (0.721) without improving AUC—evidence that loose thresholds harvest prevalence, not representation quality. Oracle-val F1 (0.741) is only marginally above always-anomaly. TCN remains slower (51 vs. 19 ms) without consistent accuracy gains.

5 Discussion & Threats to Validity

What sequence means: Protocol A orders archive members, not wall-clock events. Protocol B removes cross-member windows and duplicate lines but still uses backup-day folders rather than verified timestamps.

Window OR labels: OR aggregation manufactures high prevalence; trivial classifiers set the F1 ceiling. Ranking metrics and lift-over-prevalence are mandatory companions to F1.

Proxy labels: HTTP/keyword heuristics are not SOC truth; day shift may change keyword base rates.

Ethics: Campus telemetry stays private; any release needs redaction and authorization.

6 Conclusion

We showed that archive-order splits and OR window labels can inflate both prevalence and apparent detector quality on HUFLIT campus logs. After deduplication, source-bounded windows, and day holdout, masked TCN/Transformer

ranking collapses while PCA retains a modest AUPRC lift; TCN adds latency without a reliable benefit. Future work: true event timestamps, SOC labels, public-benchmark parity, and—separately—measured local LLM narration.

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure. In compliance with Springer Nature authorship policies, generative AI tools were used to assist in literature review verification, manuscript refinement, and English copy-editing; all core technical research, architectural designs, empirical implementations, and experimental evaluations were conducted directly by the authors.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach Based on Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019).
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., He, Z., Yu, G.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Dependable Secure Comput. 20(4), 2762–2780 (2023). <https://doi.org/10.1109/TDSC.2022.3162857>
6. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Mining Console Logs for Large-Scale System Problem Detection. In: Proc. 3rd Conf. SysML (2008).
7. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
9. Abidin, M., Aneja, J., Awadalla, H., et al.: Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219 (2024).
10. Jiang, A.Q., Sablayrolles, A., Mensch, A., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023).
11. Detmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient Finetuning of Quantized LLMs. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36 (2023).
12. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S.: AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In: Proc. Mach. Learn. Syst. (MLSys), vol. 6 (2024).
13. Ahmed, T., Ghosh, S., Bansal, C., et al.: Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In: Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), pp. 1737–1749 (2023). <https://doi.org/10.1109/ICSE48619.2023.00149>
14. Soldani, J., Brogi, A.: Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. ACM Comput. Surv. 55(3), 1–39 (2022).
15. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 355–366 (2023).
16. Moustafa, N., Slay, J.: UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. In: Proc. IEEE Mil. Commun. Inf. Syst. Conf. (MilCIS), pp. 1–6 (2015).
17. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
18. Ma, Z., Yang, J., Chen, T.H.: LLM4Log: A Systematic Review of Large Language Model-based Log Analysis. arXiv preprint arXiv:2604.16359 (2026).
19. Ma, L., Liu, J., Zhang, T., VanNostrand, P.M., Hofmann, D.M., Cao, L., Rundensteiner, E.A., Chen, J.: Krone: Hierarchical and Modular Log Anomaly Detection. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1773–1787 (2026).
20. Wu, Z., Wang, J., Zheng, L., Zhang, Y., Li, S., Chen, L.: Efficient Zero-shot and Label-free Log Anomaly Detection for Resource-constrained Systems (MaidLog). In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 671–684 (2026). <https://doi.org/10.1109/ICDE65706.2026.00056>
21. Liu, M., Wang, T., Zhao, Y., Xia, C., Zeng, Y., Tao, Y.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. (2026). <https://doi.org/10.1109/TSC.2026.3700403>
22. Dietz, S., Klede, K., Nguyen, A., Eskofier, B.M.: ContraLog: Log File Anomaly Detection with Contrastive Learning and Masked Language Modeling. arXiv preprint arXiv:2602.03678 (2026).
23. Pei, C., Cui, H., Li, J., et al.: Smart Eye: LLM-Guided Proposer-Verifier Framework for Industrial-Scale Log Anomaly Detection. In: Proc. ACM Web Conf. (WWW), pp. 7869–7879 (2026). <https://doi.org/10.1145/3774904.3792804>
24. Liao, N., Liu, Z.: Log Anomaly Detection Method Based on Transformer and Temporal Convolutional Networks. IEEE Access 13, 68547–68560 (2025). <https://doi.org/10.1109/ACCESS.2025.3561669>
25. Zhu, X., Tang, X., Wu, S., et al.: CoLA: Model Collaboration for Log-based Anomaly Detection. Proc. VLDB Endow. 18(11), 3979–3987 (2025). <https://doi.org/10.14778/3749646.3749668>